CISC3024 Pattern Recognition

AI Assignment #2

Adaptable Bayesian Neural Networks for Classification and Uncertainty Estimation

A CPU-scale, data-driven study of “Make Me a BNN” (CVPR 2024)

Student: HOI KAI CHENG

Student ID: UC325381

Report mode: Full experiment

All implementation, experiments, figures, and report composition were performed by the AI agent under the student’s prompts and choices, as required by the assignment.

1. How I Asked AI Tools to Find the Algorithm

I began by giving Cursor's AI agent the assignment PDF and the following prompt. This established the six required report sections and left the literature search, implementation, experiments, and writing to the agent.

Excerpt read at build time from the original user message:

```
d:\UM\Year4\CISC3024\AI Assignment2\AIassign2.pdf
Your AI assignment report in Word/PDF format should contain the following details (1) how you ask AI
tools to find the algorithm, (2) algorithm description, (3) how AI implements the algorithm, (4)
experiment settings and results, (5) what you have learnt from this AI assignment, and (6) a webpage
link of your source codes.
```

The agent read the one-page brief, extracted the requirement for a recent probability- or Bayes-based recognition algorithm, and used web searches to check publication venue, recency, computer-vision relevance, code availability, and CPU feasibility. It narrowed the search to three peer-reviewed candidates instead of selecting the first Bayesian paper it found.

Table 1. Candidate papers researched by the AI agent.

Candidate	Venue	Why it was considered
ABNN — "Make Me a BNN: A Simple Strategy for Estimating Bayesian Uncertainty from Pre-trained Models"	CVPR 2024	Directly Bayesian, newest candidate, post-hoc conversion of normalization layers, and uncertainty/OOD evaluation.
DDU — "Deep Deterministic Uncertainty: A New Simple Baseline"	CVPR 2023	Fast single-pass alternative using regularized features and post-hoc Gaussian discriminant analysis.
I-EDL — "Uncertainty Estimation by Fisher Information-based Evidential Deep Learning"	ICML 2023	Dirichlet evidential uncertainty with a Fisher-information objective, but a less direct BNN fit.

The agent presented these as a multiple-choice decision and recommended ABNN because it is explicitly Bayesian, was published at CVPR 2024, and permits a small direct implementation that exposes the mathematics clearly. I replied: "ABNN (CVPR 2024) and keep doing it," supplied my name and student ID, and requested publication in my CISC3024 GitHub repository.

The implementation choice was then refined for the available CPU: a CIFAR ResNet-20 rather than the paper's larger ResNet-50 and WideResNet-28×10 models, CIFAR-10 as in-distribution data, and SVHN plus CIFAR-100 as practical OOD tests. The agent selected a direct PyTorch implementation instead of hiding the mechanism behind a third-party uncertainty package.

2. Algorithm Description

2.1 From MAP to Bayesian prediction

A deterministic classifier normally keeps one parameter vector. Maximum a posteriori (MAP) training combines the data likelihood with a prior over weights. With a Gaussian prior, the prior term is the familiar L2 weight penalty:

$$\omega_MAP = arg\,min_\omega\,[-\Sigma_i \log p(y_i \mid x_i, \omega) - \log p(\omega)] \quad (1)$$

A Bayesian neural network instead predicts by marginalizing over the posterior distribution of plausible weights:

$$p(y \mid x, D) = \int p(y \mid x, \omega)\, p(\omega \mid D)\, d\omega \quad (2)$$

2.2 Bayesian Normalization Layer (BNL)

ABNN starts from a trained deterministic network and replaces its normalization layers with Bayesian Normalization Layers. For a normalized activation h_j, the paper perturbs the learned scale γ_j with independent Gaussian noise ε_j. Unit Gaussian noise on every BatchNorm of ResNet-20 collapsed the MAP features, so this reproduction uses $\varepsilon_j \sim N(0, 0.1^2)$:

$$BNL(h_j) = ((h_j - \hat{\mu}_j) / \hat{\sigma}_j)\, \gamma_j(1 + \varepsilon_j) + \beta_j,\ \ \varepsilon_j \sim N(0, 0.1^2 I) \quad (3)$$

The running statistics, affine scale, bias, and batch counter are copied from the original BatchNorm layer. The convolutional backbone and classifier are frozen; only γ and β are adapted. A new Gaussian draw on every forward pass creates different predictions without learning a full distribution for every network weight.

2.3 Multiple modes and finite inference

One local posterior mode is not enough to represent a multimodal deep-network posterior. The paper therefore starts several ABNN members from the same MAP checkpoint and changes class weights during adaptation so that members specialize differently. Our member m minimizes weighted cross-entropy with 3 seeded emphasized classes at weight 5. At inference, M members and L Gaussian draws per member approximate the Bayesian integral:

$$p(y \mid x, D) \approx (1 / ML) \Sigma_{m=1}^{M} \Sigma_{l=1}^{L} p(y \mid x, \omega_m, \varepsilon_l) \quad (4)$$

Predictive entropy $H[p] = -\Sigma_c\, p_c \log p_c$ is used as the OOD score: larger entropy means a less certain prediction and therefore a more OOD-like sample. Calibration is measured by expected calibration error (ECE); classification also reports accuracy, negative log-likelihood (NLL), and Brier score.

2.4 What the paper reports

CVPR 2024 paper findings. Franchi et al. evaluate ABNN with larger pre-trained architectures across image classification and semantic segmentation. They report that post-hoc BNL adaptation preserves the predictive properties of the deterministic network while improving uncertainty quantification with relatively small training overhead. Their analysis attributes most gains to epistemic uncertainty, as measured by OOD detection, and studies multiple posterior modes and multiple stochastic samples.

Those published conclusions are background evidence, not results measured by this assignment. Section 4 separately labels and reports our ResNet-20 values from persisted JSON; the reduced CPU study is not claimed to reproduce the paper's absolute numbers.

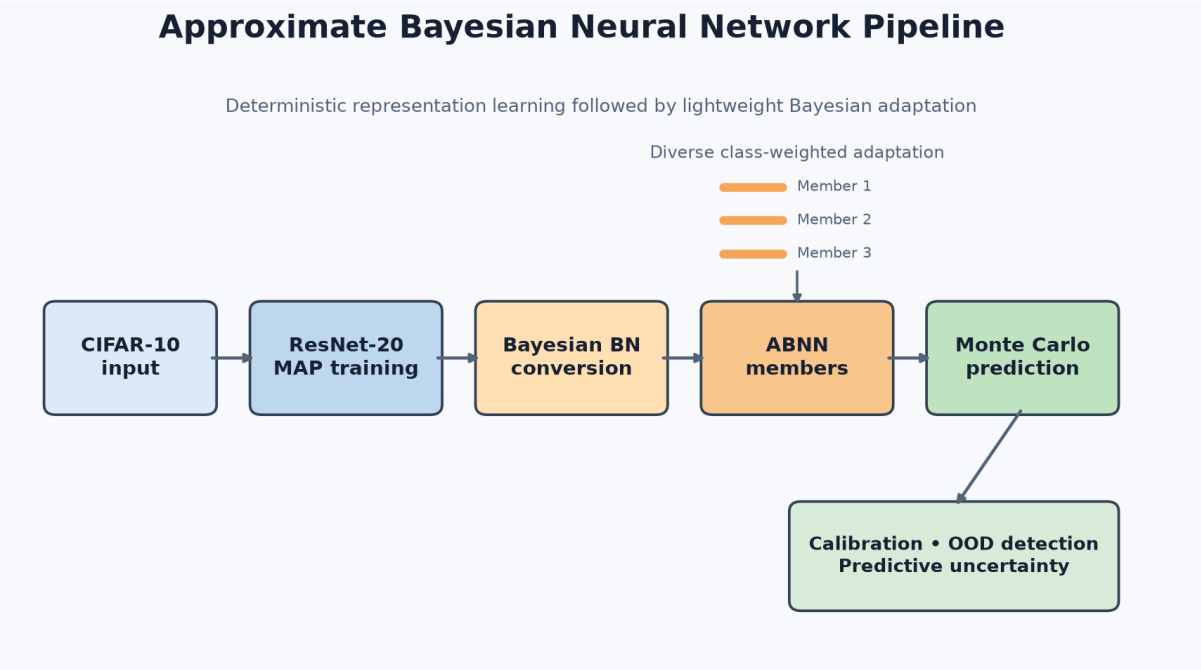


Figure 1. ABNN architecture and inference pipeline. The deterministic ResNet-20 checkpoint is converted by replacing BatchNorm with Bayesian normalization, then stochastic predictions are averaged. [artifact: architecture.png]

3. How AI Implemented the Algorithm

3.1 Implementation sequence

1. The agent encoded all approved quick/full settings and deterministic seeds, then built reproducible CIFAR-10, SVHN, and CIFAR-100 loaders.
2. It implemented a CIFAR ResNet-20 and trained the deterministic MAP checkpoint with SGD, warm-up, cosine decay, and resumable state.
3. It recursively converted every BatchNorm2d layer into BayesianBatchNorm2d while preserving state, then froze all parameters except normalization scale and bias.
4. It adapted seeded ABNN members with class-weighted cross-entropy and persisted checkpoint provenance so members cannot silently resume from a different MAP model.
5. It evaluated MAP, one-member ABNN, and all-member ABNN predictions, checking loader order and probability validity before writing JSON.
6. It generated six figures exclusively from the saved JSON and recorded source/output SHA-256 digests in a manifest.
7. It composed this DOCX from those artifacts and asked Microsoft Word to export the matching PDF.

3.2 Source layout and tests

Table 2. Main AI-generated source modules.

File	Responsibility
code/config.py	Quick/full policy and reproducible seeds
code/model.py	CIFAR ResNet-20 backbone
code/abnn.py	BNL conversion, freezing, and MC prediction
code/train_map.py	MAP training and JSON history
code/train_abnn.py	Class-diverse ABNN adaptation
code/evaluate.py	Classification, calibration, and OOD output
code/make_figures.py	Six provenance-tracked figures
code/run_all.py	Resumable end-to-end orchestration
build_report.py	Data-driven Word/PDF report

The agent followed test-driven development: each module's public behaviour was first expressed as a failing pytest, then implemented and checked against the full suite. Tests cover BNL state conversion and stochasticity, parameter freezing, finite normalized Monte Carlo probabilities, metric calculations, deterministic evaluation order, checkpoint provenance, atomic figure publication, orchestration, and report content/Word cleanup.

3.3 Real source excerpts

The report generator reads these excerpts directly from the current source files. This prevents a copied code listing from drifting away from the implementation.

Excerpt read at build time from code/abnn.py (BNL forward path):

```
    def forward(self, input: torch.Tensor) -> torch.Tensor:
        if not self.stochastic:
    # ...
        )
        scale = self.weight
        if scale is None:
    # ...
            )
        scale = scale * (1.0 + self.noise_std * torch.randn_like(scale))
        return F.batch_norm(
            input,
    # ...
            exponential_average_factor,
            self.eps,
        )
```

Excerpt read at build time from code/abnn.py (normalization-only adaptation):

```
def freeze_except_bayesian_norm(model: nn.Module) -> None:
    for parameter in model.parameters():
        parameter.requires_grad = False
    for module in model.modules():
        if isinstance(module, BayesianBatchNorm2d) and module.affine:
            module.weight.requires_grad = True
            module.bias.requires_grad = True
```

Excerpt read at build time from code/abnn.py (finite Monte Carlo averaging):

```
@torch.inference_mode()
def monte_carlo_probabilities(models, loader, samples: int, device):
    if samples < 1:
    # ...
        raise ValueError("Monte Carlo prediction requires at least one model")
    draws = []
    saved_targets = None
    for model in models:
        model.eval()
        set_stochastic(model, True)
        for _ in range(samples):
            batches, targets = [], []
    # ...
                )
    sample_probs = torch.stack(draws).numpy()
    probabilities = sample_probs.mean(axis=0)
    if not np.isfinite(sample_probs).all() or not np.isfinite(probabilities).all():
    # ...
        raise ValueError("Monte Carlo probabilities must be normalized")
    return probabilities, saved_targets.numpy(), sample_probs
```

4. Experiment Settings and Results

4.1 Settings loaded from evaluation JSON

The backbone is the source-defined CIFAR ResNet-20. All values in Table 3 are read from the evaluation artifact's config and effective run metadata; quick-mode overrides are shown rather than the unused full defaults.

Table 3. Effective experimental settings loaded from evaluation JSON.

Setting	Persisted value
Mode	full
Global seed	0
Dataset limits	train=all, cifar10_test=all, svhn_test=all, cifar100_test=all
Batch size	128
MAP epochs / learning rate	20 / 0.1
ABNN epochs / learning rate	5 / 0.01
SGD momentum / MAP weight decay	0.9 / 0.0005
ABNN members / member seeds	3 / [101, 102, 103]
Monte Carlo draws per member	3
Emphasized classes / weight factor	3 / 5.0

4.2 Training history and provenance

Table 4. MAP history loaded from map_history JSON.

Epoch	Loss	Accuracy	NLL	Brier	ECE
1	1.7133	0.4300	1.6960	0.7601	0.1388
2	1.3899	0.5072	1.5957	0.6658	0.1746
3	1.0009	0.5810	1.3464	0.5688	0.1224
4	0.8358	0.6100	1.1382	0.5300	0.1199
5	0.7353	0.6963	0.9036	0.4232	0.0700
6	0.6732	0.7235	0.8030	0.3854	0.0722
7	0.6249	0.7009	0.9149	0.4213	0.0912
8	0.5923	0.7782	0.6677	0.3137	0.0364
9	0.5581	0.7876	0.6284	0.3005	0.0492
10	0.5259	0.7568	0.7533	0.3507	0.0566
11	0.4917	0.8086	0.5669	0.2711	0.0292
12	0.4557	0.8045	0.5865	0.2786	0.0376
13	0.4261	0.8191	0.5350	0.2563	0.0245
14	0.3895	0.8092	0.5645	0.2735	0.0516
15	0.3598	0.8500	0.4417	0.2151	0.0284
16	0.3228	0.8669	0.3947	0.1922	0.0236
17	0.2893	0.8745	0.3806	0.1829	0.0278
18	0.2572	0.8847	0.3448	0.1682	0.0236
19	0.2348	0.8855	0.3361	0.1632	0.0255
20	0.2227	0.8892	0.3298	0.1595	0.0219

Persisted MAP wall-clock time: 2296.27 s.

Table 5. ABNN adaptation history loaded from abnn_history JSON.

Member	Epoch	Weighted loss	Emphasized classes
1	1	0.2725	3, 5, 9
1	2	0.2673	3, 5, 9
1	3	0.2648	3, 5, 9
1	4	0.2648	3, 5, 9
1	5	0.2638	3, 5, 9
2	1	0.2329	2, 5, 9
2	2	0.2248	2, 5, 9
2	3	0.2269	2, 5, 9
2	4	0.2296	2, 5, 9
2	5	0.2245	2, 5, 9
3	1	0.2124	1, 3, 7
3	2	0.2082	1, 3, 7
3	3	0.2096	1, 3, 7
3	4	0.2104	1, 3, 7
3	5	0.2065	1, 3, 7

Table 6. Checkpoint provenance persisted with evaluation results.

Artifact	File	SHA-256
MAP checkpoint	map_checkpoint.pt	e3aee072542b5b2fc4bb77f24fe9e175017643a96719b1e731b732fe03782b98
ABNN member 1	abnn_member_0.pt	aa0fd2dc19428943de5e5e0274d1198b3e8aca85b23df3

Artifact	File	SHA-256
		2c9a82cbec1563b334
ABNN member 2	abnn_member_1.pt	3e3f90fe3598dfe21e8dfbf8f5a5ecc0e5788a57aaa0278d1b2e10cb79ac6361
ABNN member 3	abnn_member_2.pt	01026510fc9d42b76ecbc55a6ca4f9b2508c369d939ec0b018587969b54893d7

The six figures were accepted only after their manifest matched the full source JSON digests and every PNG's SHA-256. This prevents a report from silently combining quick and full artifacts.

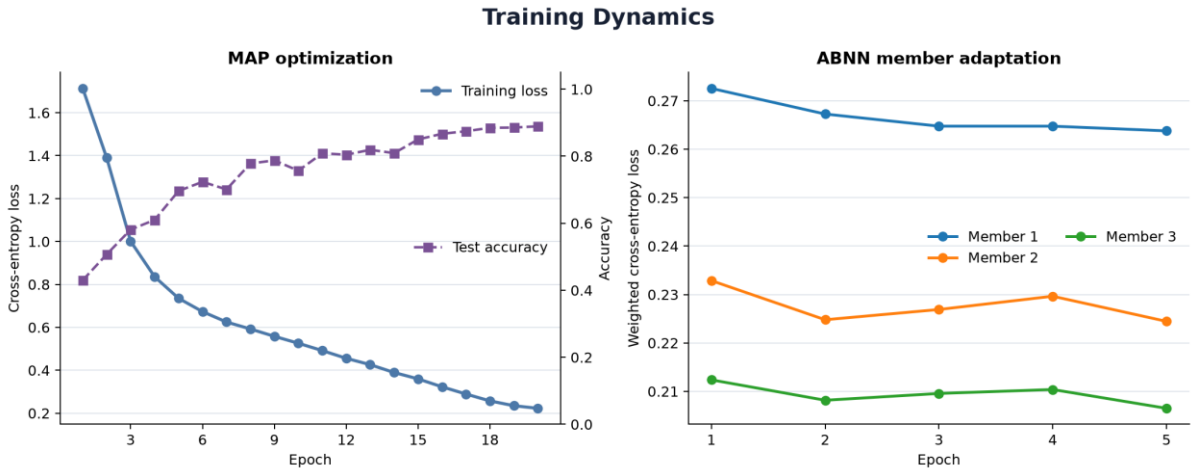


Figure 2. Training curves loaded from the persisted MAP and ABNN history JSON artifacts. [artifact: training_curves.png]

4.3 Classification and calibration

Table 7. Measured CIFAR-10 metrics loaded from evaluation JSON (accuracy higher is better; NLL, Brier, and ECE lower are better).

Method	Accuracy	NLL	Brier	ECE
MAP	0.8892	0.3298	0.1595	0.0219
ABNN (single member)	0.8675	0.3884	0.1899	0.0087
ABNN (all available members)	0.8819	0.3558	0.1736	0.0309

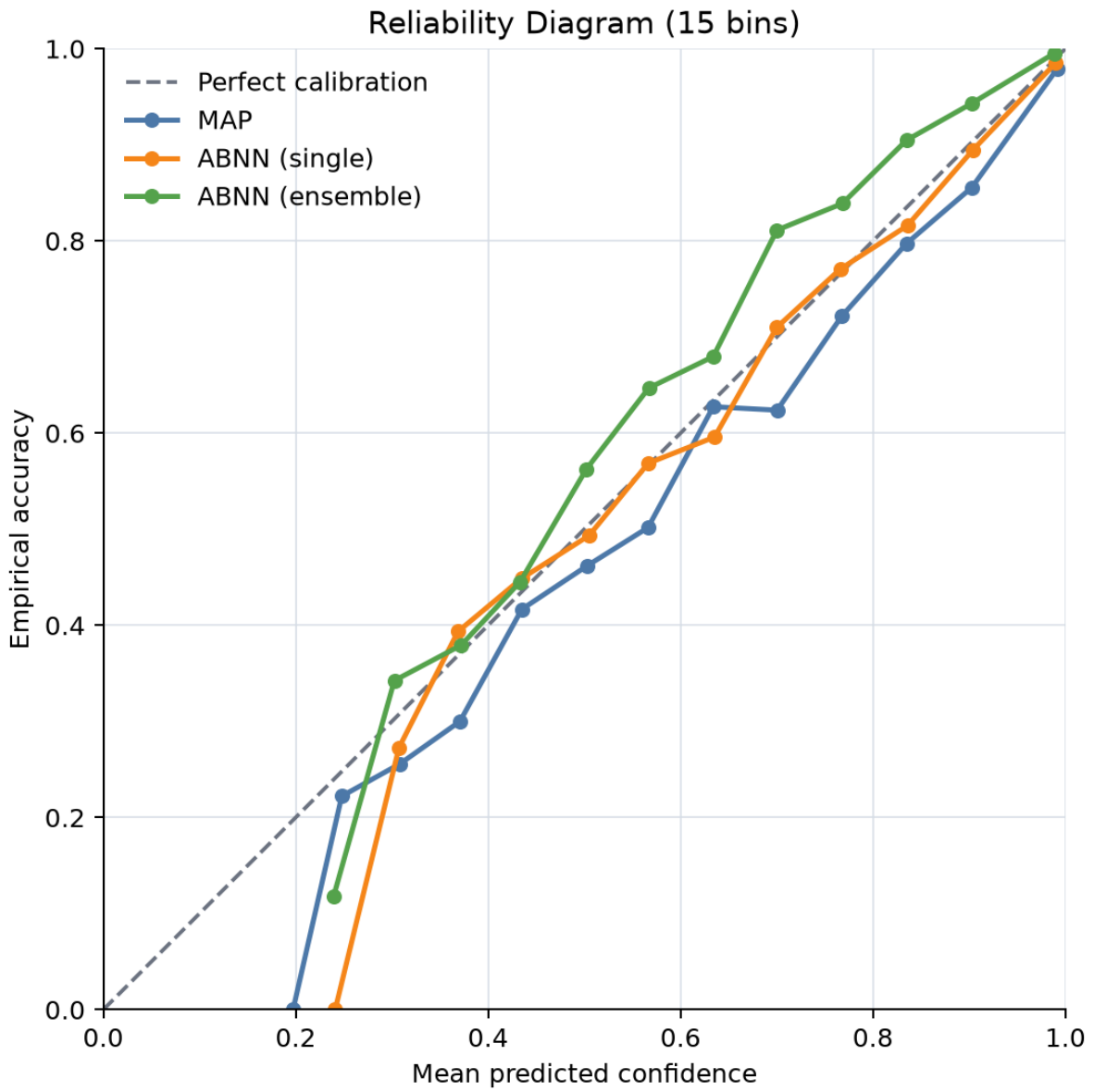


Figure 3. Reliability diagram for MAP, single-member ABNN, and the complete available-member path. [artifact: reliability.png]

4.4 OOD detection

Table 8. Predictive-entropy OOD metrics loaded from evaluation JSON (AUROC/AUPR higher is better; FPR95 lower is better).

Method	OOD set	AUROC	AUPR	FPR95
MAP	SVHN	0.8979	0.9371	0.3138
MAP	CIFAR-100	0.8461	0.8116	0.4557
ABNN (single member)	SVHN	0.8102	0.8798	0.4334
ABNN (single member)	CIFAR-100	0.8203	0.7859	0.5283
ABNN (all available members)	SVHN	0.8577	0.9058	0.3580
ABNN (all available members)	CIFAR-100	0.8440	0.8089	0.4814

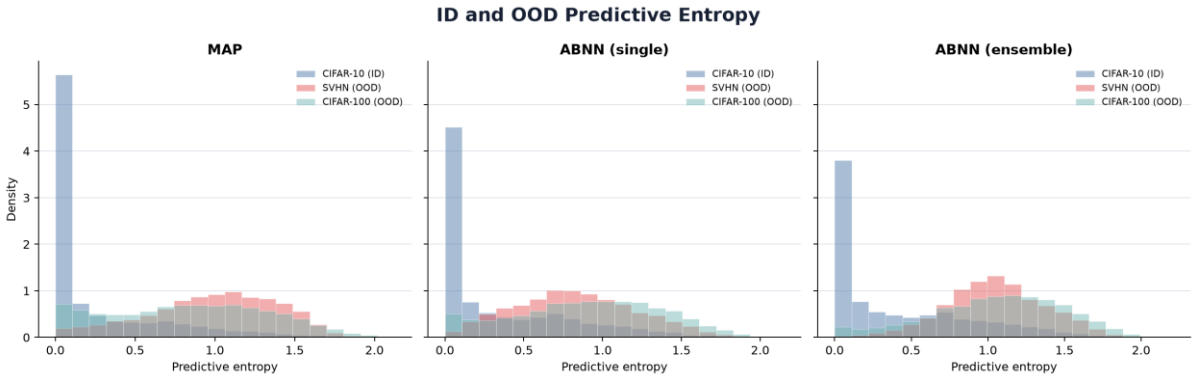


Figure 4. OOD entropy distributions for CIFAR-10 (in-distribution), SVHN, and CIFAR-100. [artifact: ood_entropy.png]

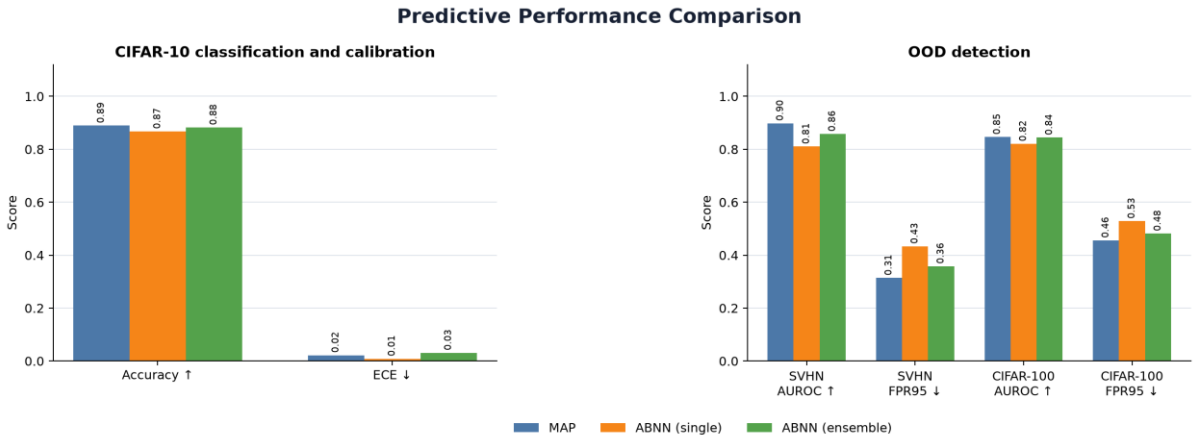


Figure 5. Metrics comparison for classification, calibration, and OOD detection. [artifact: metrics_comparison.png]

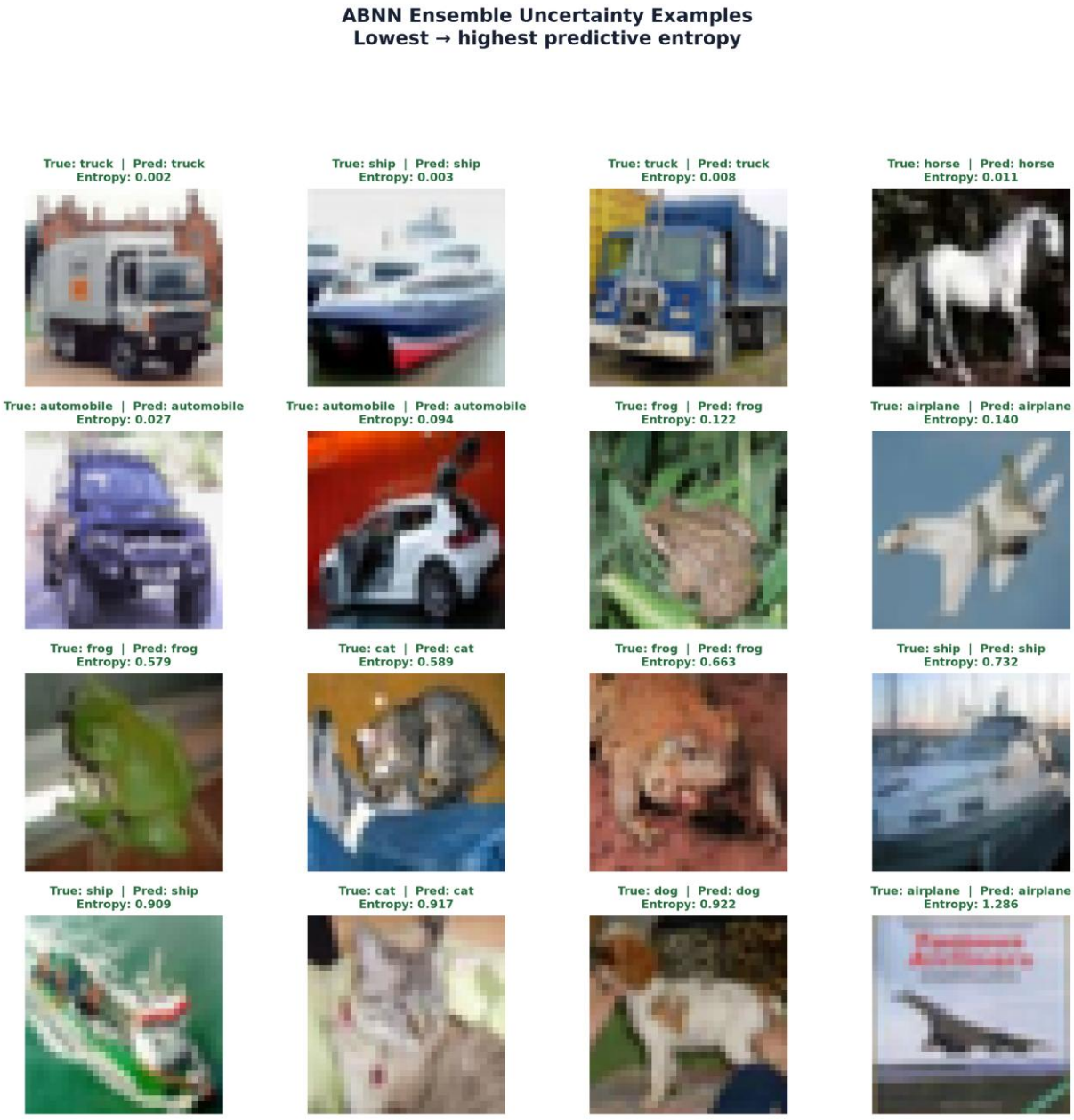


Figure 6. Uncertainty examples ordered by ABNN predictive entropy, from the least to the most uncertain saved CIFAR-10 examples. [artifact: uncertainty_examples.png]

4.5 Interpretation of measured assignment results

Measured assignment results. MAP accuracy is 0.8892, while the all-available-member ABNN path records 0.8819. Its ECE is 0.0309, compared with 0.0219 for MAP. On SVHN, entropy AUROC changes from 0.8979 (MAP) to 0.8577 (ABNN); on CIFAR-100 it changes from 0.8461 to 0.8440. These are our persisted measurements and are not values quoted from the paper.

Observed outcome. The all-available-member ABNN path underperformed MAP by 0.73 percentage points on CIFAR-10. Its entropy AUROC was also lower on both reported OOD datasets. These unexpected degradations are retained exactly as measured, with no post-hoc tuning or replacement of results. They show that this short ResNet-20 adaptation did not reproduce the large-model gains reported by the paper.

The full report uses 3 adapted members and 3 Monte Carlo draws per member. Even then, differences remain descriptive because the study uses configured MAP seed 0 and a much shorter schedule than the paper.

4.6 Limitations

- CPU model scale: CIFAR ResNet-20 is much smaller than the ResNet-50 and WideResNet-28×10 models studied in the CVPR paper, so absolute paper results are not directly comparable.
- Short schedule: the configured full run uses 20 MAP epochs and 5 ABNN adaptation epochs; this report uses 20 MAP epochs and 5 ABNN adaptation epochs and is deliberately short relative to the paper.
- One seed: the assignment uses global MAP seed 0. Without repeated training seeds, small metric differences have no confidence interval and should not be presented as statistically conclusive.
- OOD dataset choice: SVHN supplies a strong semantic/domain shift, while CIFAR-100 is visually closer to CIFAR-10. These two datasets do not represent all real deployment shifts and entropy is only one OOD score.
- Method adaptation: class-weighted ABNN members and a small normalization-based ResNet preserve the central mechanism, but this is a CPU-scale reproduction rather than the authors' exact training recipe.

5. What I Have Learnt

I learnt that a Bayesian prediction is not just a softmax confidence. The important operation is averaging predictions over plausible models or stochastic parameter samples. ABNN makes this idea practical by placing uncertainty in normalization scales rather than learning a distribution for every convolutional weight.

I also learnt why MAP, posterior modes, and stochastic draws describe different parts of the process. The MAP checkpoint supplies a strong deterministic starting point; class-weighted adaptation encourages several nearby modes; repeated Gaussian perturbations sample within each mode. The final average is therefore two-dimensional: across members and across draws.

Working with an AI agent showed me that the highest-value human input was setting constraints and making explicit choices. I selected ABNN from researched alternatives and specified the repository and identity, while the agent converted those choices into tests, code, experiments, plots, and this report. The process still required verification: probability normalization, deterministic data order, checkpoint hashes, and source-linked figures were all checked rather than trusted because they were AI-generated.

Finally, I learnt that reproducibility and honest limitations matter more than a flattering score. Every measured value in this report is loaded from JSON, code excerpts are read from source, and figures are checked against a digest manifest. The final values remain a short-schedule CPU study using configured MAP seed 0 and should be interpreted at that scale.

6. Webpage Link of the Source Codes

The direct GitHub folder containing the source code, tests, results, figures, and report is:

https://github.com/HOIKAICHENG/CISC3024/tree/main/AI%20Assignment2

https://github.com/HOIKAICHENG/CISC3024/tree/main/AI%20Assignment2

The encoded space in AI%20Assignment2 is intentional: this URL opens the assignment folder directly instead of linking only to the repository root.

References

[1] G. Franchi, O. Laurent, M. Leguéry, A. Bursuc, A. Pilzer, and A. Yao, "Make Me a BNN: A Simple Strategy for Estimating Bayesian Uncertainty from Pre-trained Models," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 12194–12203.

https://openaccess.thecvf.com/content/CVPR2024/html/Franchi_Make_Me_a_BNN_A_Simple_Strategy_for_Estimating_Bayesian_CVPR_2024_paper.html

[2] J. Mukhoti, A. Kirsch, J. van Amersfoort, P. H. S. Torr, and Y. Gal, "Deep Deterministic Uncertainty: A New Simple Baseline," CVPR, 2023, pp. 24384–24394. https://openaccess.thecvf.com/content/CVPR2023/html/Mukhoti_Deep_Deterministic_Uncertainty_A_New_Simple_Baseline_CVPR_2023_paper.html

[3] D. Deng, G. Chen, Y. Yu, F. Liu, and P.-A. Heng, "Uncertainty Estimation by Fisher Information-based Evidential Deep Learning," Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023, pp. 7596–7616. https://proceedings.mlr.press/v202/deng23b.html

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016, pp. 770–778. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[5] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, 2009 (CIFAR-10/CIFAR-100). https://www.cs.toronto.edu/~kriz/cifar.html